



STELLANTIS · GITHUB COPILOT · TOKEN OPTIMIZATION

Getting the Most Out of Every Token

A Best-Practice Guide — from Beginner to Power User

Usage-Based Billing Edition

GitHub AI Credits · effective June 1, 2026

Version 1.0 · June 2026

Contents

How to use this guide.....	3
How to prompt.....	4
Part 1 — Why this matters: the new economics	6
Part 2 — Foundations: how LLMs and agents really work	8
Part 3 — Start here: the four habits that bring 80% of the savings	9
Part 4 — Intermediate: working smarter	11
Part 5 — Power user: engineering your workflow	14
Part 6 — The Agentic Framework.....	17
Part 7 — Traps that bite (awareness, not daily decisions)	19
Part 8 — Quick reference	20
Part 9 — Future-proofing: the skills that will matter	21
Sources and further reading	22

How to use this guide

On **June 1, 2026**, GitHub Copilot moved from flat **premium requests** to **usage-based billing**. Every token you send and every token the model writes back now has a price, drawn from a shared pool of **GitHub AI Credits** (1 credit = \$0.01). The practical consequence is dramatic: the same task done carelessly versus carefully can differ in cost by **10–20×**. This guide turns that economics into a small set of concrete habits — and, happily, the habits that save credits are the same ones that improve answer quality.

It is written for **everyone** — from someone who installed Copilot last week to a power user designing multi-agent workflows. The practices are layered by difficulty, so you can adopt them in order.

Who should read what

Beginners: read Parts 1–3. The four habits in Part 3 alone capture most of the achievable savings.

Intermediate: add Part 4 for model selection, prompting, modes, and conversation hygiene.

Power users: read everything — Parts 5–6 cover workflow engineering and the traps that bite.

Everyone: keep Part 7 (Quick Reference) within reach.

One principle runs through the whole guide: as much context as required, as little as necessary. Quality and cost move together — leaner, better-guided usage is both cheaper and better.

How to prompt

Why Mastering Your Prompts Matters

When using an AI assistant like GitHub Copilot under a usage-based billing model, your prompt is your direct financial steering wheel. Writing vague or incomplete prompts forces you into unnecessary clarification loops. **Having a good prompt from the very beginning ensures you do not have to rerun the exact same query 3 times to get the right answer. (and don't waste credit)**

Every single time you rerun a query or hit "Regenerate", you are resending the entire conversation history, compounding your input token costs and burning shared AI Credits unnecessarily. Efficiency in code generation starts with precision in human language.

The Stellantis Safe Prompting Baseline

Before typing your first prompt, you must strictly align with the **Stellantis GitHub Copilot Safe Prompting and Baseline Document (CISO Document)**

[GitHub Copilot Safe Prompting Baseline](#)

.This corporate baseline mandates that your interactions with Copilot prioritize data confidentiality and repository security at all times.

The absolute golden rule of safe prompting is: **Abstract, do not disclose**. You must describe the programmatic pattern or logic goal you need, rather than detailing your real infrastructure or environment.

Prohibited Input Content (Never send these to Copilot):

- **Secrets & Credentials:** Never include passwords, active API keys, raw tokens, connection strings, or .env file values.
- **Production Logs:** Never paste real system diagnostics, pre-production/production logs, or stack traces.
- **Sensitive Identifiers:** Real customer or employee data (\$PII/PHI\$), internal architecture topologies, specific server hostnames, or cluster network maps are strictly banned.

The Anatomy of a Perfect Beginner Prompt

To get the best possible response on the first try while maintaining absolute security, structure your prompts using these four simple rules:

1. **Use Abstract Placeholders:** Replace all real asset names with generic brackets (e.g., use <Service_A>, <DB_Name>, or <Resource> instead of actual internal project names).
2. **Be Bounded and Precise:** Ask small, single-purpose questions rather than broad, open-ended conceptual queries. Specify the exact language, framework, or expected behavior up front.
3. **Inject Stop Signals:** Explicitly tell the model what *not* to do (e.g., "Stop if X", "Do not write an explanation", or "Do not refactor surrounding methods").
4. **Demand Output Trimming:** Conclude your prompt with explicit phrasing to enforce concise answers, which instantly slashes your expensive output token bills.

Everyday Habits for Beginners

- **One focused question per prompt:** Don't ask Copilot to build a feature, write tests, and document it all in one go. Break it down.
- **If you make a mistake:** If you notice you typed a real secret or internal asset name, **do not hit enter**. Completely clear the input box, rewrite it with safe placeholders, and then submit.
- **Always validate the output:** Copilot is a probability machine, not a deterministic engineer. Every single line of code generated must be reviewed for security compliance, injection safety, and accuracy before you attempt to commit it to a repository.

Part 1 — Why this matters: the new economics

What changed on June 1, 2026

Before, Copilot ran on **premium requests (PRUs)**: one interaction equalled one flat unit, no matter how much context you sent or how long the answer was. From June 1, every interaction is billed by the **actual tokens processed**. There are three kinds:

- **Input tokens** — everything you send: your prompt, pasted code, attached files, open editor tabs.
- **Output tokens** — everything the model writes back. These are the expensive ones.
- **Cached tokens** — context reused across messages in the same conversation, billed at a steep discount.

Two other changes landed at the same time:

- **Code completions and Next Edit Suggestions stay free** and unlimited — they never touch the pool.
- **No more silent fallback.** When the budget is exhausted, Copilot simply stops until the next cycle, unless an admin explicitly allows paid overage. There is no automatic downgrade to a cheaper model.

The shared pool — and why your habits matter

Credits are **pooled at the billing-entity level**, not handed out per seat. An organization with 100 Business seats gets one shared pool of 190,000 credits, not 100 separate buckets. Power users can draw more when they need it, and lighter users offset them. The flip side: one careless session spends from **everyone's** budget.

The cookie-jar model

Picture one shared jar. Every seat you buy drops credits in; everyone — developer, manager, data scientist, designer — scoops from the same jar. Heavy, wasteful usage empties it faster for the whole team, and once it is empty, work can stop. That is why individual discipline is a collective benefit.

What is free and what costs

Free — never touches the pool	Billed — draws from the pool
Code completions (inline ghost text)	Copilot Chat (Ask)
Next Edit Suggestions	Agent mode in the IDE
	Copilot CLI
	Copilot Coding Agent (cloud)
	Copilot Code Review (CCR)
	Copilot Spaces / Spark
	Third-party coding agents

The heavy hitters

Long agent sessions and frontier models consume by far the most. A quick chat with a lightweight model can cost a fraction of a credit; an unsupervised, multi-file agent run on a top-tier model can cost hundreds. Most monthly spikes trace back to a single agent session.

The real stake: the same task, three ways

Consider one ordinary task — refactor a 50-line function to add proper null handling. The cost depends almost entirely on **how** you ask:

- **The wasteful way (~15 credits):** a powerful model, the entire 500-line file pasted as context (~15,000 input tokens), a request for a full-file rewrite with a detailed explanation (~3,000 output tokens), in a brand-new chat with no cache reuse.
- **The disciplined way (~1.2 credits):** a versatile model, only the relevant function pasted (~1,500 input tokens), a request for just the changed lines with no commentary (~500 output tokens).
- **Disciplined + cached (~0.8 credits):** the same, but a few messages into a focused thread, so the context is already cached (~10× cheaper input).

Approach	Cost / task	Tasks per month (Example for 1,900 cr)
Wasteful — powerful model, full file, verbose, new chat	~15 cr	~125
Disciplined — versatile model, scoped, concise	~1.2 cr	~1,580
Disciplined + cached context	~0.8 cr	~2,375

The same monthly budget does 12–19× more work depending on habits. Closing that gap — without working less — is what the rest of this guide is about.

Part 2 — Foundations: how LLMs and agents really work

You don't need machine-learning expertise, but a correct mental model makes every later habit obvious instead of arbitrary.

An LLM is a probability machine

At its core, a model predicts the next token from the text it has been given — nothing more. It has **no memory of you between calls** and no hidden knowledge of your repository. It only ever sees the text in its **context window**. Everything you do to guide it is, ultimately, arranging that text well.

The harness, the model, and “no magic — just text”

When you work with an agent — VS Code Chat, Copilot CLI, the cloud Coding Agent, Claude Code, Codex — three pieces are in play: **you and your project**; the **harness** (the agent that assembles prompts, reads files, and calls tools); and the **LLM** itself. The LLM is **stateless**: on each loop it is handed a fresh pile of text and asked for the single next step. There is no magic — it is text in, text out. Knowing this is why prompts, instructions, files, and tool outputs all matter equally: they are all just text being arranged for the model.

The four kinds of tokens

Token type	What it is
Input	What you send: prompt, files, context, open tabs
Output	What the model writes back
Cached read	Context reused from earlier in the same conversation
Cache write	First-time storage of context (Anthropic / Claude models)

Output is the expensive one.

Context windows and “context rot”

Every model has a maximum context window, but filling it is neither free nor helpful. Models bias toward the **beginning and end** of the context and lose information in the middle (“lost in the middle”). Once the window is more than ~50% full, they increasingly favour the most recent tokens (**recency bias**). The practical rule is blunt: just because you **can** fill the window doesn't mean you **should**.

Relevance beats volume

More tokens ≠ better answers. Provide only what is relevant. (Background reading on the effect: [Context Rot — producttalk.org](https://producttalk.org/context-rot).)

The compound-error problem

Agents work in multi-step loops, and small error rates multiply fast. Even at **99% reliability per step**, a 50-step workflow only lands around **60%** correct; at 95% per step it collapses to roughly **8%**. This is the deep reason to scope tasks, add deterministic guardrails, and divide work into phases: every derailed run wastes credits on debugging, CI minutes, and review cycles. Reliability is a cost-control strategy.

Part 3 — Start here: the four habits that bring 80% of the savings

If you do nothing else, do these four. They are simple, they compound, and together they account for most of the savings you can realistically achieve.

Habit 1 — Scope your context

Why it matters most: input scales linearly with what you send, on every single message. A 500-line file is ~7,000 tokens; the relevant 30-line function is ~400 — an **~18× reduction** you collect again and again.

- Paste only the function or block in question — not the whole file.
- Reference files by name and let Copilot read the specific range itself (`read_file` with line bounds).
- Keep only 5–6 relevant editor tabs open — VS Code sends open tabs as context.
- Attach folders and files deliberately, not reflexively.

Habit 2 — Match the model to the task

Why it matters: model tier is the single biggest multiplier on per-token cost. Picking the right tier is the cheapest lever you have.

- **Default to a versatile model** (Sonnet 4.6 / GPT-5.3-Codex) for daily coding and agent work.
- **Use a lightweight model** (Haiku 4.5 / GPT-5 Mini) for formatting, lookups, doc updates, and simple refactors.
- **Reserve a powerful model** (Opus 4.8 / GPT-5.5) for genuinely hard architecture, planning, or debugging — and only when the versatile tier visibly fails.
- **Or pick Auto** and let Copilot route the task — it also earns a 10% model-cost discount on paid plans.

Names are familiar; tiers shift

Don't assume a model's tier from its name. "Haiku 4.5", for instance, is a versatile-class model, not the cheapest option — for truly cheap inference, use a genuine lightweight model. Model names and prices change often, so check the live pricing table (Part 4) rather than relying on memory.

Maximize your consumption

For complex tasks, instruct a high-reasoning model to conduct deep research and generate a structured blueprint in a Markdown (.md) file. Then, pass that Markdown file to a fast, versatile model (like Claude 3.5 Sonnet) to execute and implement the code based exactly on the blueprint.

Habit 3 — Demand concise output

Why it matters: output is 5–8× more expensive than input. The long, beautifully formatted explanation Copilot loves to write costs real money on every call.

- "Just the changed lines, no explanation."
- "Yes or no?"

- “One-sentence summary of the issue, then the fix.”
- For code review: “flag issues only, skip the praise.”
- Don’t ask for a **full-file rewrite** when you changed three lines.

Habit 4 — Stay in the same conversation

Why it matters: the first message pays full input price; every later message in the same thread reuses the prefix as **cached tokens at ~10× lower cost**.

- One conversation per work topic; ask follow-ups in the same thread.
- Don’t close and reopen the chat for a “clean slate” — it’s expensive.
- Reference an earlier answer instead of re-asking from scratch.

Warning One chat for one subject

One chat for one task, start a new chat for a new task, don't carry over useless context which stays in cache from a precedent task to a new task which doesn't benefit for this context carry over

Best things to do

1. **Set a sensible default model** — Auto, or a versatile model like Sonnet 4.6.
2. **Stop pasting whole files** — share only the relevant block.
3. **Add “be concise”** to your prompting reflexes.
4. **Keep one thread per task** to bank cache discounts.
5. **Glance at your usage once** so you know what normal looks like (Part 4).

Part 4 — Intermediate: working smarter

The model lineup and what each is for

Prices are per 1 million tokens (~750k words). Treat this as a snapshot — names and rates change frequently, so confirm against the live GitHub Docs pricing page. The pattern is stable even when the names aren't: lightweight < versatile < powerful, and output always costs multiples of input.

Model	Task area	Input	Cached	Cache write	Output
GPT-5.5	Powerful reasoning, debugging	\$5.00	\$0.50	—	\$30.00
Claude Opus 4.8	Deep reasoning & debugging	\$5.00	\$0.50	\$6.25	\$25.00
Claude Sonnet 4.6	General-purpose coding & agents	\$3.00	\$0.30	\$3.75	\$15.00
GPT-5.3-Codex	Agentic software development	\$1.75	\$0.18	—	\$14.00
Claude Haiku 4.5	Fast, lightweight tasks	\$1.00	\$0.10	\$1.25	\$5.00
GPT-5 Mini	Code completion & writing	\$0.25	\$0.03	—	\$2.00

Which model for which task

Task	Recommended	When to use
Everyday coding, debugging	Sonnet 4.6 / GPT-5.3-Codex	Your default for daily work
Simple / mechanical (formatting, lookups, docs)	GPT-5 Mini / Haiku 4.5	Cheap and fast
Hard architecture, deep refactor, planning	Opus 4.8 / GPT-5.5	Only when the versatile tier fails
Code review, no deep issues expected	Haiku 4.5 / GPT-5 Mini	Lightweight review pass
When in doubt	Auto	Routes for you, +10% discount (paid)

Auto mode and the 10% discount

From June, **Auto mode** detects task intent and picks the model for you — a reasoning model for architecture, planning, and debugging; a mid-tier model for implementation; a smaller model for refactoring, docs, and repetitive tasks. On paid plans, using Auto in Chat, CLI, or the cloud agent earns a **10% discount on model costs**. It is the sensible lazy default.

Write prompts that don't waste a round-trip

A vague prompt causes clarification loops, and each loop is a paid round-trip. Three moves prevent most of them:

- **Be precise.** Say exactly what to change and where — “fix the null check in processOrder, line 45”, not “fix that thing we discussed.”
- **Add stop signals.** “Stop if X.” “Don't refactor anything else.” This keeps the agent from wandering into expensive territory.
- **Add known context up front.** Name the files, folders, or docs it needs so it doesn't burn tokens hunting for them.

Pick the right mode for the job

- **Completions / Next Edit Suggestions** — free; let them handle routine typing.
- **Ask (Chat) mode** — for questions and explanations; the cheapest interactive surface.
- **Agent mode** — for multi-step actions across files; powerful, but it reads files and dispatches work, so it's overkill for “what's the syntax for X?”.

Manage your conversations

- **One thread per task** to keep cache hits.
- Use `/clear` (or a new thread) when you **genuinely** switch task — stale context is paid context and degrades quality via context rot.
- **Compact long sessions cautiously.** Use `/compact` for Compacting summarizes history to save tokens, but it can drop details you still need. Use it deliberately, not reflexively.
- **Delete requests that didn't help** so they stop riding along as context.

Tips

You can prompt after `/compact` to precise the agent what to keep in memory and what he should delete

Don't pay twice for the same mistake

- **Don't habitually regenerate.** “Regenerate” is a new full-cost request, not a free retry — refine the prompt instead.
- **Avoid full-file rewrites for small changes;** ask for diffs or changed lines only.
- **Paste text, not screenshots.** Images are tokenized and billed; paste the error text when you can.

Watch your consumption — three ways

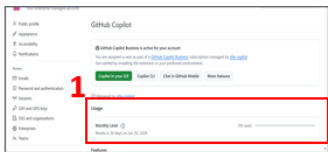
Where	How	What you see
github.com	Copilot → Usage (Billing Overview)	Pooled usage, preview bill, history
VS Code	Live indicator, bottom-right status bar	Credits used this cycle, in real time
CLI	<code>/usage</code> in the terminal	Live credit / token counter for the session

1

🌐

On the GitHub website

Open the [Copilot](#) usage page from your account.



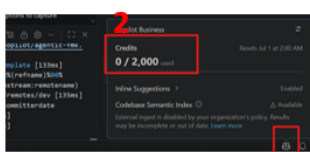
github.com → Copilot → usage

2

</>

In your IDE (VS Code)

Live indicator in the bottom-right status bar.



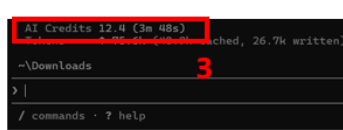
status bar · bottom-right

3

>_

From the CLI

Run the usage command in your terminal.



`/usage`

Check it regularly

Spikes almost always trace back to a single agent session. A quick weekly glance at the Billing Overview is enough to catch a runaway run before it drains your pool.

Part 5 — Power user: engineering your workflow

At this level you stop optimizing single prompts and start engineering the **system around the agent**. Two levers dominate everything: **model choice** (Part 4) and **context** — and context is where the real upside lives.

Divide and conquer: Research → Plan → Implement

Instead of one giant agent run, split the work into phases, each with the right model and a clean handoff:

- **Research.** “I want to change X — which files are relevant?” A strong reasoning model maps the surface and produces a plan input.
- **Plan.** Turn that into a precise spec — reasoning model, tight scope, explicit constraints.
- **Implement.** Hand the precise spec to a cheaper, faster model (or a fleet of subagents) to make the changes.

Why it saves: each phase carries only the context it needs, avoids one bloated session, and catches mistakes before they compound into an expensive debugging run.

Provide deterministic guardrails

The cheapest way to stop wasted credits is to **fail fast, deterministically**. With unit tests, a buggy change produces failing tests and a quick correction. Without them, the agent layers buggy change on buggy change, then burns a long debugging session — plus CI minutes and review cycles — on what is now an incident.

- **Tests, linters, type checks, and security scans** as pre-tool / CI gates.
- **Scope checks to where they apply** — e.g., run a Terraform tag check only on `.tf` files.
- **Keep a human in the loop** on agent actions for anything risky or irreversible.

Persistent instructions: your always-on guidance

A small, human-written instructions file (`.github/copilot-instructions.md` or `AGENT.md`) is sent with **every** request. Use it for the things that should never be re-explained:

- The non-negotiables of the project (conventions, forbidden patterns, required structure).
- A running log of recurring agent misses — treat each miss like an incident to be logged and fixed.
- Short statements that trim output (“be concise”).

Rules for the instructions file

Keep it small — it’s always-on, so every token is paid on every request. **Don’t generate it with AI**. And **iterate, maintain, and even recreate it often** as the project and the agent’s behaviour evolve.

Check our agentic Framework !

Our agentic framework is already doing these advice by putting deterministic guardrail, having persistent instruction and Research → Plan → Implement based on the task complexity [stla-copilot/agentic-fmk: Framework for copilot agentic usage](https://stla-copilot.com/agentic-fmk)

Customize the agent without bloating context

GitHub Copilot exposes several customization layers. The cost trick is progressive disclosure: load detail only when it is relevant, instead of carrying everything in every prompt.

Mechanism	What it is	Loads when	Best for
<code>copilot-instructions.md</code> <code>/AGENT.md</code>	Always-on standards & non-negotiables	Every request	Conventions, agent-miss log, output trimming
Scoped instructions	Conditional standards by file-path pattern	When matching files are touched	Path-specific rules
<code>.prompt.md</code> files	Reusable, manually invoked workflows	On <code>/command</code>	Scaffolding, reviews, migrations
Custom agents	Role-based personas with curated tools	On selection / handoff	Plan → Implement → Review chains
Skills	Conditional capabilities, described cheaply	When relevant (offered to the model)	Domain capabilities (API patterns, runbooks)
MCP servers	Third-party tool / data integrations	When a tool is called	Issues, databases, external systems
Copilot memory	Automated small always-on notes	Every request, across surfaces	Lightweight cross-surface context

The key cost idea: skills and scoped instructions describe themselves in a few tokens and load their full body only when needed, rather than sitting in every prompt. Custom agents work best as the outer wrapper around a workflow; combine them with instructions (standards) and skills (capabilities).

Subagents: offload task-specific context

A **subagent** runs a focused task in its own context — say, “find my feature” across many documents — and returns only a **summary** to the main session. The main thread stays small while the expensive exploration happens out of band. Powerful, but each subagent is its own billed session, so scope them deliberately.

Learn from your own history

Treat agent configuration like engineering. Review where credits and time go, then fix the patterns. Copilot CLI’s `/chronicle` (alongside the usage dashboards) can surface findings like “you spent most of your tokens running a frontier model for plain implementation” or “nine rapid agent sessions were your single most expensive event of the month” — and then you adjust your defaults.

Use Agents debugs log

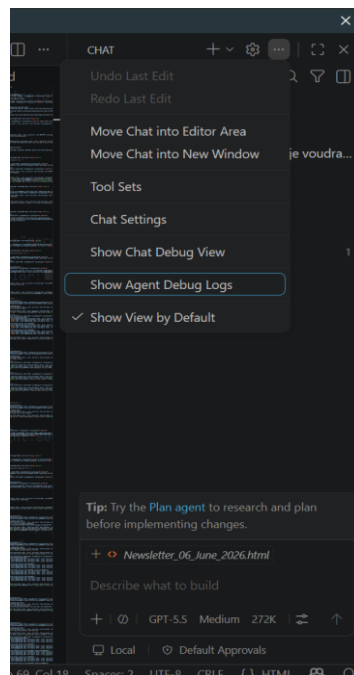
Chronology & Execution Tracking

- **Event Timeline:** Tracks every action performed by the agent, second by second.
- **Tool Calls:** Displays exactly which scripts, commands, or APIs the agent chose to execute.
- **Context & Errors:** Highlights what information was included in the prompt and captures any errors or warnings (crucial for troubleshooting custom agents or external MCP servers).

Token Consumption Metadata

You can precisely monitor token usage:

- **Input Tokens (Prompt Tokens):** The total tokens sent to the model, including your query, system instructions, and retrieved context (such as open files or @workspace search results).
- **Output Tokens (Completion Tokens):** The tokens generated by the model for its response or next execution step.
- **Total Tokens:** The combined sum of input and output tokens.



A recurring real-world finding

Reserve the most powerful model for the rare session that truly needs multi-step architectural reasoning, and run everyday implementation on a mid-tier model. A lopsided input/output ratio usually means you are paying premium prices just to read context — exactly the work a cheaper model handles fine.

Part 6 — The Agentic Framework

[stla-copilot/agentic-fmk: Framework for copilot agentic usage](#)

An unconfigured Copilot prompt easily bloats the context window with redundant or unnecessary information. The Agentic Framework remedies this by applying architectural economy directly to the workspace:

1. Progressive Disclosure (Context Isolation)

Instead of stuffing every Stellantis coding standard into the context window on every message—which would saturate your input tokens—the framework only loads critical, "always-on" landmine rules by default (Security, Observability, and Development Gates).

- Heavy, language-specific frameworks (e.g., Java SpringBoot, Python, TypeScript-backend, Terraform) are loaded **on-demand only when the active file path or task dictates it**.
- Thanks to this isolated progressive disclosure, core instruction templates have shrunk significantly (for instance, the *Constitution* skill footprint dropped from 352 lines down to ~155 lines).

2. Enforced Terse Output ("Caveman Mode")

The framework overrides the model's natural tendency to write verbose, beautifully polished, yet expensive explanations. It forces the assistant to use ultra-concise answers, bullet points, and one-line confirmations without standard introductory pleasantries. Because output tokens are 5 to 8 times more expensive than input tokens, this intervention yields instant financial savings on every single turn.

3. Shell-Output Compression (RTK)

The framework vendors a specialized binary utility called **RTK** natively inside the repository (.rtk/ folder for Windows). Whenever an agent runs local terminal tools (unit tests, linters, or build scripts), RTK works alongside native editor configurations to **compress command line output by 60% to 90%** before injecting it back into the context window. You no longer pay premium token rates for massive, uncompressed console logs.

4. Delegated Subagents and Opt-In Tasks

The framework breaks down complex workflows into isolated token-saving pathways:

- **Isolated Code Reviews:** Heavy structural reviews are handled by a dedicated, isolated subagent (powered by the cost-effective Claude Haiku 4.5) that is **dispatched only if you explicitly opt-in and accept the turn**.
- **Delegated Exploration:** For large-scale multi-file search operations, the framework spawns a read-only *Explore* subagent. This subagent scans the repository out-of-band and passes back a concise text summary to the main thread, successfully keeping your primary conversation's cached context small and clean.

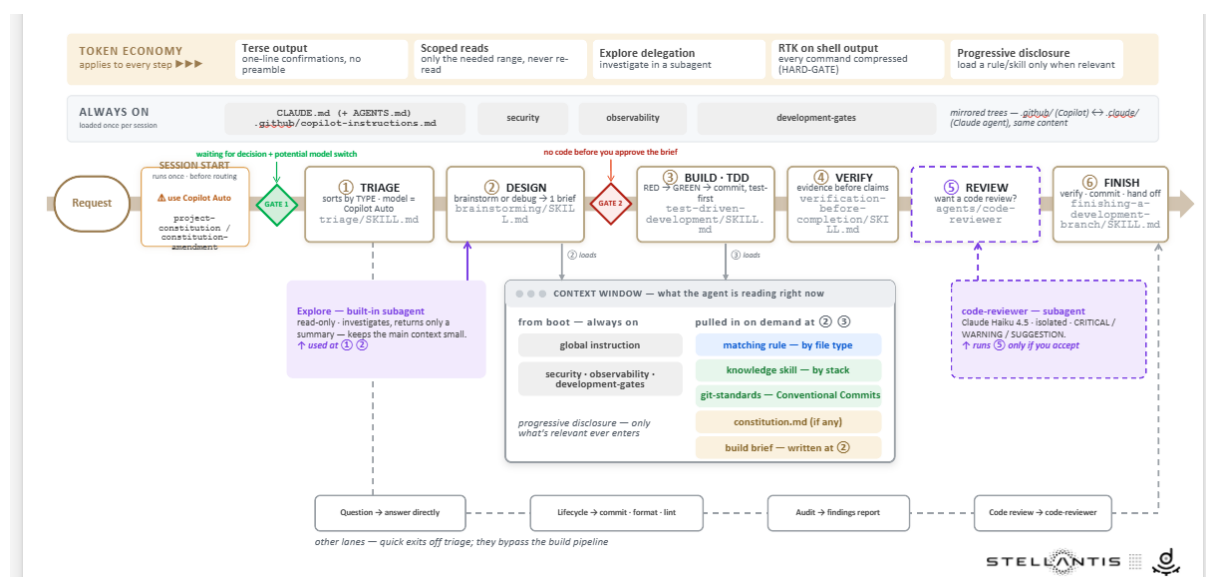
The Routing Workflow: A Built-In Cost Guardrail

The single most expensive mistake an engineer can make with AI agents is triggering a compounding error loop, where an unguided agent stacks bug upon bug, wasting hundreds of credits attempting to debug its own code. The Agentic Framework mitigates this by mandating a strict routing lane:

- **No Code Without a Brief:** The agent cannot write or refactor code on a whim. It must first generate a single build brief (specification + plan) inside docs/briefs/ under a strict gate. This cleanly separates the high-context planning phase (ideal for high-tier reasoning models) from the mechanical execution phase.
- **Test-Driven Development (TDD) Gate:** The framework forces a test-first approach. The agent must write a failing unit test *before* making implementation changes. If a code change breaks existing structures, the local test fails deterministically, and the agent stops to correct it immediately, preventing an expensive, uncontrolled multi-file debugging spiral.

Triage → Design → Build (TDD) → Review

- **No Code Without a Brief:** The agent cannot write or refactor code on a whim. It must first generate a single build brief (specification + plan) inside docs/briefs/ under a strict gate. This cleanly separates the high-context planning phase (ideal for high-tier reasoning models) from the mechanical execution phase.
- **Test-Driven Development (TDD) Gate:** The framework forces a test-first approach. The agent must write a failing unit test *before* making implementation changes. If a code change breaks existing structures, the local test fails deterministically, and the agent stops to correct it immediately, preventing an expensive, uncontrolled multi-file debugging spiral.



Part 7 — Awareness, not daily decisions

These don't require a behaviour change every day, but if you don't know them they can surprise you — sometimes badly.

- **Cloud agent sessions are the single biggest cost source.** GitHub itself cites multi-hour autonomous sessions as the reason for the billing change. One unsupervised frontier-model run can burn hundreds of credits — scope tasks, limit file access, and review intermediate output.
- **Copilot code review costs twice** — AI Credits and GitHub Actions minutes, because it runs on hosted runners by default. Use self-hosted runners where available, and don't trigger reviews on every draft push.
- **No fallback when the budget runs out.** Copilot stops entirely until the next cycle (or until an admin allows paid overage). There is no automatic downgrade.
- **MCP server outputs are billed as input.** Heavy servers — database queries, large API responses — inject big payloads into context. Filter upstream when you can.
- **Images cost tokens.** A pasted screenshot is tokenized and billed; paste text instead where possible.
- **Long-context surcharges.** Some models add a surcharge above a token threshold — trim context before you hit it.
- **Your usage is collective.** Every habit affects the shared pool, not just your seat.

Part 8 — Conclusion

The one-screen checklist

Everyone

- Default to Auto or a versatile model.
- Demand concise output.
- One thread per task; scope your context.
- Glance at your usage regularly.

Intermediate

- Precise prompts with stop signals and known context up front.
- Right mode for the job; right model for the task.
- `/clear` on a real task switch; don't habitually regenerate.
- Paste text, not screenshots.

Power user

- Research → Plan → Implement, with the right model per phase.
- Tests, linters, and scans as deterministic gates.
- A small, human-written instructions file as agent-miss log and output trimmer.
- Progressive disclosure: skills and scoped instructions over always-on bloat.
- Scope subagents and cloud runs; review `/chronicle` regularly.

Five things to start doing today

1. **Choose the right model** for the right task.
2. **Provide clear guidance** in your prompts.
3. **Research → Plan → Implement.**
4. **Provide deterministic guardrails** (tests, linters, security scans).
5. **Maintain a concise, human-written** `copilot-instructions.md` — use it as an agent-miss log and to trim outputs.

Part 9 — Future-proofing: the skills that will matter

The billing change is also a nudge toward a more durable way of working. Three traits will keep paying off well beyond credit savings:

Build your analytical skills

Writing code was never the core value of a developer — it was the analytical skill and the ability to become proficient in any domain quickly. Being able to tell an agent precisely what to do, in the language of the domain, becomes the single most valuable capability.

Apply good architecture

Domain-Driven Design, hexagonal architecture, CQRS, event-driven design — these make a codebase easier for agents to discover and give stronger guardrails, which in turn avoids the excess fix / debug / error sessions that quietly burn credits.

Iterate on prompts and agent configs

Approach AI agents with an engineering mind. Keep configurations fresh, treat agent misses like incidents to be diagnosed, and review your usage regularly so improvement is driven by data, not guesswork.

Sources and further reading

This guide synthesizes GitHub's official documentation, GitHub's engineering blog, an internal GitHub token-optimization workshop, and community guides. Prices and model names change frequently — always confirm against the live GitHub Docs pricing page.

Official GitHub

- **Usage-based billing (orgs & enterprises)** — docs.github.com
- **Usage-based billing (individuals)** — docs.github.com
- **Models and pricing for GitHub Copilot** — docs.github.com
- **Best practices for using GitHub Copilot** — docs.github.com
- **Prompt engineering for Copilot Chat** — docs.github.com
- **Copilot is moving to usage-based billing** — github.blog
- **Want better AI outputs? Try context engineering** — github.blog

Community guides & deep dives

- **github-copilot-token-optimization** — github.com/olivomarco
- **awesome-copilot (instructions, agents, skills)** — github.com/github
- **GitHub Copilot Customization Handbook** — copilot-academy.github.io
- **Context Rot (the “lost in the middle” effect)** — producttalk.org

Token prices and model availability shown in this guide are a June 2026 snapshot and will change. The live GitHub Docs pricing page is always authoritative.